

Contents

1	Introduction	3
1.1	Introducing the game engine series	3
1.2	What is a game engine?	3
1.2.1	Game engines and assets	3
1.3	Designing out game engine	4
1.3.1	Division of labor of the game engine (Game Engine Systems)	5
1.4	Entry point	6
1.4.1	Code	6

Chapter 1

Introduction

1.1 Introducing the game engine series

- The game engine series is intended to present what is a game engine, how it is constructed and a hands on tutorial on how to build a game engine.
- A game engine is an engine to make real-time, interactive and sometimes 3d rendering applications.
- There are many way in which a game engine can be constructed. EA and Rockstar games are constructed using a game engine that they have in house. It is a piece of software that is enormous. The game engine we are going to build is going to be simpler to illustrate the concept and make something cool.
- Our engine will be able to render 3d graphics, not just for games but also for any application that requires interactive, 3d rendering.
- Building an engine takes years of a team of people, so the engine we are going to be building will be simpler but it will take long as well.
- Take this playlist on youtube as a full course on this topic since it will have diagrams, explanations, code demonstrations and live demos.
- In addition we will have a repository on github and set up the whole project as a real workable project with profesional functionality and collaborators.

1.2 What is a game engine?

- A game engine has many definitions, some people define it as a tool to build games quickly, unity and unreal are examples of game engines. These commercially available game engines present themselves as a set of tools (level editor) or a platform for building and interactive application, be that a game or any application that has those requirements.
- For some companies the product is not a game engine, but an actual game. Rockstar games provides the gran theft auto game, not the engine used to make it. In these cases they have their own game engines build that are only used for their games. They don't release it, they only release the game.

1.2.1 Game engines and assets

- A game engine is a platform to build applications for real-time rendering such as video games, simulations, etcetera.

- This platform transforms data from one format into another. We take the data from disk or ram, and transform the data into something else that shows up on our screens. Optionally, we can build in functionality to interact with said data.
- A game engine, is not a hard-coded application that does something. A game engine, is a piece of software that reads files in, transforms them, and displays them on the screen and adds interactive capabilities.
- Game engines are extremely data oriented. The game engine is not expected to create data based on nothing, we are loading existing files and transforming it into a visual format that we can see on our screen.
- What are these files that the game engine takes as input and how we can make them? These files are called assets. Note that they do not have to be a physical file, but essentially data represented in some form.
- In addition to a game engine being able to display the files taken as input (assets), it typically provides a way of making said files (assets).
- Assets may be done in something like photoshop or blender, but at the end of the day, assets need to be in a format that the game engine can actually receive and process.
- Usually the assets are not in JPG, or PNG format, usually the format used for the assets are a format especially designed for that game engine. Be it textures, levels, etcetera. The asset creator will also be responsible of transforming the asset in whatever third party format (be it JPG, PNG, PDF, etc) into the game engine format.
- The game engine reads those assets at run time and presents something on the screen based on that.
- There are many game engine systems, which for our purposes are any means of abstraction used in our game engine. For example: serialization, rendering, file manipulation, multiplatform abstraction tools, etcetera. This will be explained later so don't worry if you don't understand.
- The main take away: a game engine helps facilitate the general task of making a rendering application, without it each game would have to be hardcoded with everything each level needs, every object annotated exactly, with a game engine we can just run it with some parameters and we can concentrate on the implementation of the idea rather than the details of the game engine. Games today are so immense and complex that if we did not have game engines we would end up writing all code from scratch every time, and it would be a waste of time. Think of GTA, the levels and details, if they would need to implement this every time they release a new game it would be impossible. So engineers can build the tools so that people that are not computer experts can implement their ideas.
- Game engines are a huge topic so don't expect it to be a straight forward topic.
- The language we are going to use is C++. Step by step.

1.3 Designing out game engine

- Game engines are hard and complex pieces of software. Most large companies hire already made game engines, and if they decide to code it themselves it is a large and complicated task that takes a lot of resources.
- Since the only person building this is The Chernob, then we are going to cut corners.
- So the idea will be that we will write out our basic code and as we progress we revisit the already made code and add more features and fix it if necessary. Don't expect perfect code in this tutorial.
- There are some compromises, for example we are not going to have a level editor and other such features.

1.3.1 Division of labor of the game engine (Game Engine Systems)

The game engine will do some things, and will not do other things. Defining what our game engine does is the first step. Now we will define the division of labor, what components will do what, what will the game engine do and what it will expect already done. So the following components are defined:

- **Entrypoint:** the entry point is what is expected for the game engine to do once it is launched. For example the main function, the entry point controls what happens when we launch our entry point.
- **Application layer:** code that deals with application life-cycle and events, what executes the code that our game will use, what handles the events, what keeps our application running inside the run loop? Windows being resized? Mouse and keyboard? All of this is handled by this component.
- **Window layer:** a window is something that really only exists on desktop systems, our entire application will be in the window. This layer will handle input and events:
 - **Inputs & events:** the event manager will handle input and events. This handles things such as being notified when a key is pressed, checking the current mouse position when a click is triggered, etcetera. Basically checking the state of the device and doing things when an event is captured.
- **Renderer:** it is the components that actually renders graphics on the screen, this is going to be the biggest and most complicated system. The renderer is not the game engine, this is just a part. The renderer is much easier to implement if all the other systems mentioned thus far are implemented already.
- **Render API Abstraction:** supporting multiple rendering APIs, for our purposes we are going to use OpenGL. We could use DirectX but The Cherno already has a series on OpenGL, and OpenGL is much simpler. It is important that if we decide to go with another renderer API we don't need to re-write much of the already existing code. Make this as much render API agnostic. Somethings you are going to have to implement specifically for the renderer, but the idea is to abstract the render API as much as possible.
- **Debugging support:** building the application in such a way that it is easy to see what is happening. For example a logging system, a profiling and performance system, etcetera. We want to put debugging code into our code that does this and not rely on external tools. Such as a timing function, logging function, profiling function, etcetera.
- **Scripting language:** instead of having to do things in C++ all the time, we can create or use a scripting language so that people can write their own scripts without thinking about stuff such as memory.
- **Memory systems:** custom allocators and memory tracking. Memory is a scarce resource and in terms of performance we need optimized memory methods of allocating, deallocating, and tracking.
- **Entity-component system (ECS):** way to create game objects and modularize them, using systems to give them characteristics, etcetera. For example: we can create an object, and attach a script to it so that it can bounce around the room, or a physics script that permits gravity to the object, etcetera.
- **Physics:** a physics solution, adding physics to objects.
- **File I/O, Virtual File System (VFS).**
- **Build systems:** this is to ensure that everything is in the right format so that no unnecessary data transformation happens at run time, but rather on build time. This also gets capabilities such as being able to edit the assets, hit ctrl-s and the game engine already running imports the asset again in order to check if it works as desired.

1.4 Entry point

- An application's entry points is the point in which the application runs.
- Our engine is a dll file, which means when we attach the code to our game engine application, the entry point will be the point in which the application starts the engine.
- The game engine is supposed to be attached to the end user application. So we don't expect it to have a main function, because the main function will be in the end user application.

1.4.1 Code

- First we create the following folder and files:

```
src
  Hazel
    Application.cpp
    Application.h
    Core.h
    Entrypoint.h
  Hazel.h
sandbox
  src
    SandboxApp.cpp
```

Now the commented first code in this lecture explained.